# Dynamic Programming

Week 6 – Algorithms

Korea University Sejong Campus, Dept. of Computer Science & Software

# Part 1. Foundations of Dynamic Programming

# Recursive Solutions: Light and Shadow

**When recursion works well:**

- Subproblem inputs sum to at most the original input size
- Merge sort, quick sort, factorial, graph DFS

**When recursion is catastrophic:**

- Subproblem inputs sum to **more** than the original input
- Fibonacci numbers, matrix chain multiplication

> The key issue: **overlapping subproblems** cause exponential redundant computation.

# What is Dynamic Programming?

**Dynamic Programming (DP)** avoids redundant computation by:

1. Solving **small subproblems** first
2. **Storing** (memorizing) their solutions
3. Using stored solutions to solve **larger subproblems**
4. Building up to the **original problem**

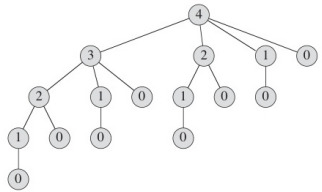
**Two requirements for DP:**

| Property | Description |
|---|---|
| **Optimal Substructure** | Optimal solution contains optimal solutions to subproblems |
| **Overlapping Subproblems** | Recursive solution re-solves the same subproblems repeatedly |

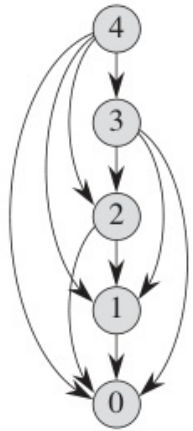
# DP vs. Divide-and-Conquer

```
DaC                     DP
   A                     A
  / \                   / \
 B   C                 B   C
/|   |\               /|\ /|\
D E   F G             D  E F  G
                        (shared!)
```

- DaC: subproblems are **independent**
- DP: subproblems **overlap** – solve once, reuse
- DaC: top-down | DP: bottom-up (small → large)




Tree →




DAG (DP)

# Two DP Implementation Strategies

### Top-Down: Memoization

- Use recursion but cache results in a table
- Only solve subproblems that are actually needed

### Bottom-Up: Tabulation

- Fill a table iteratively from smallest subproblems
- Systematic – solves all subproblems in order

Top-Down (Memoization)

```
fib(n):
  if memo[n] exists
    return memo[n]
  memo[n] = fib(n-1)
            +fib(n-2)
  return memo[n]
```

Bottom-Up (Tabulation)

```
f[1] = f[2] = 1
for i = 3 to n:
  f[i] = f[i-1]
       + f[i-2]
return f[n]
```

# Example 1: Fibonacci Numbers

**Definition:** $f(n) = f(n-1) + f(n-2)$, with $f(1) = f(2) = 1$

**Naive recursion:**

```
fib(n):
    if n = 1 or n = 2: return 1
    return fib(n-1) + fib(n-2)
```

**Call tree for fib(7):** Enormous redundancy!

```
              fib(7)
            /        \
        fib(6)       fib(5)        ← fib(5) computed twice
        /    \        /   \
    fib(5) fib(4)  fib(4) fib(3)   ← fib(4) computed 3 times
      ...     ...      ...     ...      ← fib(3) computed 5 times!
```

**Time complexity:** $O(2^n)$ – exponential!

# Fibonacci: Memoization vs. Bottom-Up DP

**Memoization (Top-Down):**

```
f[1] = f[2] = 1; all others = 0

fib(n):
    if f[n] ≠ 0: return f[n]
    f[n] = fib(n-1) + fib(n-2)
    return f[n]
```

**Bottom-Up DP (Tabulation):**

```
fibonacci(n):
    f[1] = f[2] = 1
    for i = 3 to n:
        f[i] = f[i-1] + f[i-2]
    return f[n]
```

| i | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| f[i] | 1 | 1 | 2 | 3 | 5 | 8 | 13 |

# Example 2: Matrix Path Problem

**Problem:** Given an $n \times n$ matrix of positive integers, find the path from top-left $(1, 1)$ to bottom-right $(n, n)$ that **maximizes** the sum of visited cells.

**Constraint:** Only move **right** or **down** (no left, up, or diagonal).

| | | | |
|---|---|---|---|
| 6 | 7 | 12 | 5 |
| 5 | 3 | 11 | 18 |
| 7 | 17 | 3 | 3 |
| 8 | 10 | 14 | 9 |

**Recurrence:** $c[i, j] = m_{ij} + \max(c[i-1, j],\ c[i, j-1])$

Boundary: $c[i, 0] = c[0, j] = 0$

# Matrix Path: DP Table

```
matrixPath(n):
    for i = 0 to n: c[i, 0] = 0
    for j = 1 to n: c[0, j] = 0
    for i = 1 to n:
        for j = 1 to n:
            c[i, j] = m[i,j] + max(c[i-1, j], c[i, j-1])
    return c[n, n]
```

| c | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 |
| **1** | 0 | 6 | 13 | 25 | 30 |
| **2** | 0 | 11 | 16 | 36 | 54 |
| **3** | 0 | 18 | 35 | 39 | 57 |
| **4** | 0 | 26 | 45 | 59 | **68** |

# Part 2. Classic DP Problems

# LCS: Problem Definition

**Longest Common Subsequence (LCS)**

- A **subsequence** is obtained by deleting zero or more characters without changing the order of remaining characters
- A **common subsequence** of X and Y appears in both strings
- The **LCS** is the longest such common subsequence

**Example:**

- X = `ABCBDAB`, Y = `BDCABA`
- Common subsequence: `BCA` (length 3)
- **LCS**: `BCBA` (length **4**)

**Goal:** Find the **length** of the LCS (and optionally the sequence itself).

# LCS: Recurrence Relation

Let $X_m = \langle x_1, x_2, \ldots, x_m \rangle$ and $Y_n = \langle y_1, y_2, \ldots, y_n \rangle$.

Define $c[i, j]$ = length of LCS of $X_i$ and $Y_j$.

$$c[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j \\ \max(c[i-1, j],\ c[i, j-1]) & \text{if } i, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

**Intuition:**

- If the last characters **match**: extend LCS of shorter prefixes by 1
- If they **don't match**: take the better result from dropping either character

# LCS: Pseudocode

```
LCS(m, n):
    for i = 0 to m: C[i, 0] = 0    // empty Y
    for j = 0 to n: C[0, j] = 0    // empty X
    for i = 1 to m:
        for j = 1 to n:
            if x[i] = y[j]:
                C[i, j] = C[i-1, j-1] + 1
            else:
                C[i, j] = max(C[i-1, j], C[i, j-1])
    return C[m, n]
```

**Time:** $\Theta(mn)$ – fill an $m \times n$ table, each cell in $O(1)$.

**Space:** $\Theta(mn)$ for the table (can be optimized to $O(\min(m, n))$).

# LCS: Step-by-Step Example

X = `ABCBDAB` , Y = `BDCABA`

| c | | B | D | C | A | B | A |
|---|---|---|---|---|---|---|---|
| | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **A** | 0 | 0 | 0 | 0 | **1** | 1 | 1 |
| **B** | 0 | **1** | 1 | 1 | 1 | **2** | 2 |
| **C** | 0 | 1 | 1 | **2** | 2 | 2 | 2 |
| **B** | 0 | 1 | 1 | 2 | 2 | **3** | 3 |
| **D** | 0 | 1 | **2** | 2 | 2 | 3 | 3 |
| **A** | 0 | 1 | 2 | 2 | **3** | 3 | **4** |

# 0-1 Knapsack Problem

**Problem:** Given $n$ items, each with weight $w_i$ and value $v_i$, and a knapsack of capacity $C$, maximize total value without exceeding capacity.

**Example:** Capacity $C = 10$ kg

| Item | Weight | Value |
|---|---|---|
| 1 | 5 kg | 10 |
| 2 | 4 kg | 40 |
| 3 | 6 kg | 30 |
| 4 | 3 kg | 50 |

**Subproblem:** $K[i, w]$ = max value considering items $1 \ldots i$ with capacity $w$.

**Answer:** $K[n, C]$

# Knapsack: Recurrence

For each item $i$ and capacity $w$:

$$K[i,w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ K[i-1,w] & \text{if } w_i > w \text{ (item too heavy)} \\ \max(K[i-1,w],\ K[i-1,w-w_i] + v_i) & \text{otherwise} \end{cases}$$

**Two choices for item $i$:**

- **Don't take it:** value = $K[i-1,w]$
- **Take it:** value = $K[i-1,w-w_i] + v_i$ (make room, add value)

```
Knapsack(n, C):
    for i = 0 to n: K[i, 0] = 0
    for w = 0 to C: K[0, w] = 0
    for i = 1 to n:
        for w = 1 to C:
            if w[i] > w: K[i,w] = K[i-1, w]
            else: K[i,w] = max(K[i-1,w], K[i-1,w-w[i]] + v[i])
    return K[n, C]
```

# Knapsack: Worked Example

Items: (5kg, $10), (4kg, $40), (6kg, $30), (3kg, $50). Capacity = 10.

| K[i,w] | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| **1** (5,10) | 0 | 0 | 0 | 0 | 0 | 10 | 10 | 10 | 10 | 10 | 10 |
| **2** (4,40) | 0 | 0 | 0 | 0 | 40 | 40 | 40 | 40 | 40 | 50 | 50 |
| **3** (6,30) | 0 | 0 | 0 | 0 | 40 | 40 | 40 | 40 | 40 | 50 | 70 |
| **4** (3,50) | 0 | 0 | 0 | 50 | 50 | 50 | 50 | 90 | 90 | 90 | **90** |

**Optimal value:** $K[4, 10] = 90$ (items 2 + 4: 4kg + 3kg = 7kg, $40 + $50 = $90).

**Time:** $O(nC)$ – pseudo-polynomial (depends on capacity value, not input size).

# Coin Change Problem

**Problem:** Given coin denominations $d_1 > d_2 > \cdots > d_k = 1$ and amount $n$, find the **minimum number of coins** to make change for $n$.

**Greedy can fail!** Example: coins = {16, 10, 5, 1}, amount = 20.

- Greedy: 16 + 1 + 1 + 1 + 1 = **5 coins**
- Optimal: 10 + 10 = **2 coins**

**DP Recurrence:**

$$C[j] = \min_{d_i \leq j} \{C[j - d_i] + 1\}$$

Base case: $C[0] = 0$.

# Coin Change: Algorithm and Example

```
DPCoinChange(n, d[1..k]):
    C[0] = 0
    for j = 1 to n: C[j] = infinity
    for j = 1 to n:
        for i = 1 to k:
            if d[i] ≤ j and C[j - d[i]] + 1 < C[j]:
                C[j] = C[j - d[i]] + 1
    return C[n]
```

**Example:** coins = {16, 10, 5, 1}, amount = 20

| j | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | … | 16 | … | 20 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C[j] | 0 | 1 | 2 | 3 | 4 | 1 | 2 | 3 | 4 | 5 | 1 | … | 1 | … | **2** |

**Answer:** $C[20] = 2$ (two 10-coin). **Time:** $O(nk)$.

# Floyd-Warshall: All-Pairs Shortest Paths

**Problem:** Find shortest paths between **all pairs** of vertices in a weighted directed graph.

**Key idea:** Incrementally allow intermediate vertices $\{1, 2, \ldots, k\}$.

**Recurrence:**

$$d_{ij}^{(k)} = \min\left(d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right)$$

- $d_{ij}^{(0)} = w_{ij}$ (direct edge weight, or $\infty$ if no edge)
- $d_{ii}^{(0)} = 0$

**Interpretation:** Is it shorter to go from $i$ to $j$ **through vertex** $k$, or without it?

# Floyd-Warshall: Algorithm

```
FloydWarshall(W, n):
    // W[i,j] = edge weight (infinity if no edge)
    D = W                       // initialize D^(0)
    for k = 1 to n:             // add vertex k as intermediate
        for i = 1 to n:
            for j = 1 to n:
                D[i,j] = min(D[i,j], D[i,k] + D[k,j])
    return D
```

**Time:** $O(n^3)$ – three nested loops over $n$ vertices.

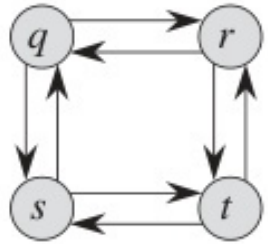
**Space:** $O(n^2)$ – the distance matrix.

**Note:** Each iteration $k$ considers all paths that may pass through vertices $\{1, 2, \ldots, k\}$ as intermediates.

# Floyd-Warshall: Example

```
Initial D^(0):
|    1    2    3    4
|-------------------------
1|   0    3   inf   7
2|  inf   0    2   inf
3|   5   inf   0    1
4|   2   inf  inf   0
```

```
After k=1,2,3,4: D^(4):
|    1    2    3    4
|-------------------------
1|   0    3    5    6
2|   5    0    2    3
3|   3    6    0    1
4|   2    5    7    0
```




Each cell $D[i, j]$ now holds the shortest distance from $i$ to $j$.

# Edit Distance (Levenshtein Distance)

**Problem:** Given two strings $X[1 \ldots m]$ and $Y[1 \ldots n]$, find the minimum number of single-character operations to transform $X$ into $Y$.

**Operations** (each costs 1):

- **Insert** a character
- **Delete** a character
- **Replace** a character

**DP Recurrence:**

$$E[i,j] = \begin{cases} j & \text{if } i = 0 \\ i & \text{if } j = 0 \\ E[i-1, j-1] & \text{if } X[i] = Y[j] \\ 1 + \min(E[i-1,j],\ E[i,j-1],\ E[i-1,j-1]) & \text{otherwise} \end{cases}$$

- $E[i-1,j]+1$: delete $X[i]$

# Edit Distance – Example

Transform "**kitten**" → "**sitting**" (m=6, n=7):

| E | ε | s | i | t | t | i | n | g |
|---|---|---|---|---|---|---|---|---|
| **ε** | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| **k** | 1 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| **i** | 2 | 2 | 1 | 2 | 3 | 4 | 5 | 6 |
| **t** | 3 | 3 | 2 | 1 | 2 | 3 | 4 | 5 |
| **t** | 4 | 4 | 3 | 2 | 1 | 2 | 3 | 4 |
| **e** | 5 | 5 | 4 | 3 | 2 | 2 | 3 | 4 |
| **n** | 6 | 6 | 5 | 4 | 3 | 3 | 2 | 3 |

# DP Problem-Solving Strategy

**Step-by-step approach to any DP problem:**

1. **Define the subproblem** – What does $OPT[i, j, \ldots]$ represent?
2. **Find the recurrence** – How does $OPT[\cdot]$ relate to smaller subproblems?
3. **Identify base cases** – What are the trivial/boundary values?
4. **Determine computation order** – Fill table bottom-up (small to large)
5. **Extract the answer** – Where in the table is the final solution?
6. (Optional) **Trace back** – Reconstruct the actual solution, not just its value

# Summary of DP Problems

| Problem | Subproblem | Recurrence | Time |
|---|---|---|---|
| Fibonacci | $f[i]$ | $f[i] = f[i-1] + f[i-2]$ | $O(n)$ |
| Matrix Path | $c[i,j]$ | $c[i,j] = m_{ij} + \max(c[i-1,j], c[i,j-1])$ | $O(n^2)$ |
| LCS | $c[i,j]$ | match: $c[i-1,j-1]+1$; else: $\max(c[i-1,j], c[i,j-1])$ | $O(mn)$ |
| 0-1 Knapsack | $K[i,w]$ | $\max(K[i-1,w], K[i-1,w-w_i]+v_i)$ | $O(nC)$ |
| Coin Change | $C[j]$ | $\min_{d_i \leq j}(C[j-d_i]+1)$ | $O(nk)$ |
| Floyd-Warshall | $d_{ij}^{(k)}$ | $\min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ | $O(n^3)$ |

# Key Takeaways

- **DP** solves optimization problems by combining solutions to overlapping subproblems
- Requires **optimal substructure** and **overlapping subproblems**
- Subproblems have an **implicit order** – solve small ones first
- Two strategies: **memoization** (top-down) and **tabulation** (bottom-up)
- DP vs. Greedy: DP considers **all** subproblem combinations; Greedy makes **one** local choice
- DP vs. DaC: DP **reuses** subproblem solutions; DaC subproblems are **independent**

> "Those who cannot remember the past are condemned to repeat it." – George Santayana (and also Dynamic Programming)

# Q & A

codingchild@korea.ac.kr